

MANIPAL SCHOOL OF INFORMATION SCIENCES

MANIPAL

(A constituent unit of MAHE, Manipal)

FIRST SEMESTER, MASTER OF ENGINEERING
COMPUTER SCIENCE AND ENGINEERING

AME 5151 - Applied Linear Algebra Lab

Mid-Semester Lab Examination

Saturday, 10-October-2026

Duration: 120 Minutes Total Marks: 30

Instructions:

- Use `numpy` to solve the problems. Do not use `np.mean`, `np.var` or `np.std` in your implementation of Question 3 (you may use them in tests).
- Write unit tests using the `unittest` module. Marks for tests are indicated in each question.
- Write well-documented code (docstrings with argument and return descriptions).
- Submit a single Python file containing the code and tests for all problems. It must run with `python <registration_number>.py`.
- Vector indices are 1-based and ranges are inclusive in the mathematical notation, i.e. $\mathbf{x}_{2:5} = (x_2, x_3, x_4, x_5)$. NumPy arrays are 0-based.
- Use `float64` throughout and a tolerance (`np.allclose` or `assertAlmostEqual`) whenever you compare floating-point results.

1. **[CO1 - BL3. 8 Marks]**
   Let $\mathbf{x} = (1, 4, 9, 16, 25, 36)$, stored as a 1-D NumPy array of shape $(6,)$, and let

$$\mathbf{D} = \begin{bmatrix} -1 & 0 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 \\ 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

   (a) Construct $\mathbf{D}$ *programmatically* (no hard-coded matrix literal) for a general size $n \times (n-1)$, then compute $\mathbf{x}^{\mathrm{T}}\mathbf{D}$. 3 Marks.

   (b) Write a function that returns the subvector $\mathbf{x}_{i:j}$ using the convention above, and use it to obtain $\mathbf{x}_{2:5}$. 1 Mark.

(c) Compute $\mathbf{d} = \mathbf{x}_{2:6} - \mathbf{x}_{1:5}$ and verify numerically that $\mathbf{d} = \mathbf{D}^{\mathrm{T}}\mathbf{x}$ (equivalently, $\mathbf{x}^{\mathrm{T}}\mathbf{D}$ from part (a)). 2 Marks.

(d) Write unit tests for parts (a)–(c). 2 Marks.

2. **[CO1 - BL3. 7 Marks]**
Let $\mathbf{X} \in \mathbb{R}^{6\times 768}$ hold six token embeddings as its rows. Generate it with `np.random.default_rng(0).standard_normal((6, 768))`.

(a) Compute $\mathbf{D}^{\mathrm{T}}\mathbf{X}$ by matrix multiplication, using the matrix $\mathbf{D}$ from Question 1. State the shape of the result. 7 Marks.

3. **[CO2 - BL4. 15 Marks]**
Analyze and examine the details of the following algorithm.

Layer normalization (shown below) rescales an activation vector to zero mean and (approximately) unit variance, then applies a learned element-wise scale and offset. Algorithm 1, shown below, describes it for a single vector.

Assume $d_{\mathrm{e}} = 768$ and $\varepsilon = 10^{-5}$.

Algorithm 1: $\hat{\mathbf{e}} \leftarrow$ `layer_norm`$(\mathbf{e}, \boldsymbol{\gamma}, \boldsymbol{\beta}, \varepsilon)$

Input	$: \mathbf{e} \in \mathbb{R}^{d_{\mathrm{e}}}$,	neural network activations.
Output	$: \hat{\mathbf{e}} \in \mathbb{R}^{d_{\mathrm{e}}}$,	normalized activations.
Parameters	$: \boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^{d_{\mathrm{e}}}$,	element-wise scale and offset.
Hyperparameter	$: \varepsilon > 0$,	small constant for numerical stability.
1 $m \leftarrow \sum_{i=1}^{d_{\mathrm{e}}} \mathbf{e}[i]/d_{\mathrm{e}}$		scalar mean
2 $v \leftarrow \sum_{i=1}^{d_{\mathrm{e}}} (\mathbf{e}[i] - m)^2/d_{\mathrm{e}}$		scalar variance, $1/d_{\mathrm{e}}$ normalization
3 **return** $\hat{\mathbf{e}} = \dfrac{\mathbf{e} - m\mathbf{1}}{\sqrt{v + \varepsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}$,		$\odot$ denotes element-wise multiplication. $\mathbf{1}$ is the all-ones vector.

(a) Structure the code of `layer_norm(···)` in NumPy. 10 Marks.

(b) Trace unit tests that check the following properties of your implementation (choose tolerances carefully, since $\varepsilon > 0$):
(i) agreement with a reference formula written using `np.mean` and `np.var`;
(ii) with $\boldsymbol{\gamma} = \mathbf{1}, \boldsymbol{\beta} = \mathbf{0}$: output mean ≈ 0 and output variance $\approx v/(v + \varepsilon)$;
5 Marks.